LAPORAN UTS PRAKTIKUM

UJIAN TENGAH SEMESTER

API PENYEWAAN KENDARAAN (REST + GRAPHQL)

Mata Kuliah	Pemrograman Berbasis Framework / API Development
Jenis Penilaian	Ujian Tengah Semester (UTS) Praktikum
Judul Tugas	Desain dan Implementasi REST API + GraphQL pada Sistem Penyewaan Kendaraan
Nama	M. ZELQY ADRIAN
NIM	TI21240011

1. Pendahuluan

Laporan ini disusun sebagai pemenuhan tugas Ujian Tengah Semester (UTS) pada mata kuliah Pemrograman Berbasis Framework. Objek yang dikerjakan adalah sebuah sistem penyewaan kendaraan yang dibangun sebagai layanan backend, sehingga seluruh pengujian dilakukan melalui permintaan API, bukan melalui antarmuka pengguna. Sistem menyediakan dua bentuk antarmuka komunikasi sekaligus, yaitu REST API sebagai jalur utama dan GraphQL API sebagai jalur alternatif yang lebih fleksibel.

Pengembangan aplikasi modern menuntut pemisahan yang jelas antara lapisan antarmuka, lapisan logika bisnis, dan lapisan penyimpanan data. Sistem ini menerapkan pola tersebut secara konsisten. Seluruh permintaan HTTP ditangani pada lapisan penyajian, seluruh aturan bisnis ditempatkan pada lapisan layanan, dan seluruh akses basis data dilakukan melalui ORM pada lapisan data. Pemisahan ini membuat setiap bagian dapat diuji dan dikembangkan secara terpisah.

Secara fungsional, sistem mencatat data pelanggan, data kendaraan, transaksi penyewaan, serta transaksi pembayaran yang terkait. Setiap penyewaan menghubungkan satu pelanggan dengan satu kendaraan, dan setiap penyewaan dapat memiliki lebih dari satu transaksi pembayaran. Kebutuhan inilah yang kemudian diterjemahkan ke dalam skema basis data relasional dengan relasi satu ke banyak (one-to-many) antar entitas.

Ruang lingkup pekerjaan UTS ini mencakup perancangan endpoint, pembuatan aturan validasi masukan, perancangan skema basis data, penulisan logika layanan, penyediaan endpoint GraphQL, hingga pengujian fungsional menyeluruh. Seluruh data contoh menggunakan tiga digit terakhir NIM, yaitu 011, agar keterkaitan antara hasil pekerjaan dan identitas mahasiswa dapat diverifikasi langsung pada respons API.

Kode sumber utama yang terdiri atas berkas endpoint, validasi masukan, lapisan layanan, model basis data, dan skema GraphQL ditampilkan dalam bentuk tangkapan layar editor agar mudah dicocokkan dengan berkas aslinya. Berkas pendukung seperti skrip seed dan skrip pengujian tidak ditampilkan karena tidak memuat logika utama aplikasi.

2. Teknologi dan Arsitektur Sistem

Sistem diimplementasikan di atas runtime Node.js versi 24 dengan bahasa TypeScript. Pemilihan TypeScript bertujuan agar kesalahan tipe data dapat terdeteksi lebih awal sebelum program dijalankan, sehingga kualitas kode lebih terjaga. Proses eksekusi menggunakan tsx, yang memungkinkan berkas TypeScript dijalankan langsung tanpa tahap kompilasi terpisah, sekaligus mendukung mode watch untuk pengembangan.

Lapisan HTTP menggunakan Express sebagai kerangka kerja web, sedangkan lapisan GraphQL menggunakan Apollo Server versi 5 yang dipadukan melalui paket integrasi resmi untuk Express 4. Sisi basis data menggunakan Prisma ORM dengan SQLite sebagai mesin basis data. Pemilihan SQLite bersifat strategis untuk kebutuhan UTS karena tidak memerlukan instalasi server basis data terpisah maupun konfigurasi pengguna dan kata sandi, namun tetap mempertahankan kemampuan relasional yang lengkap.

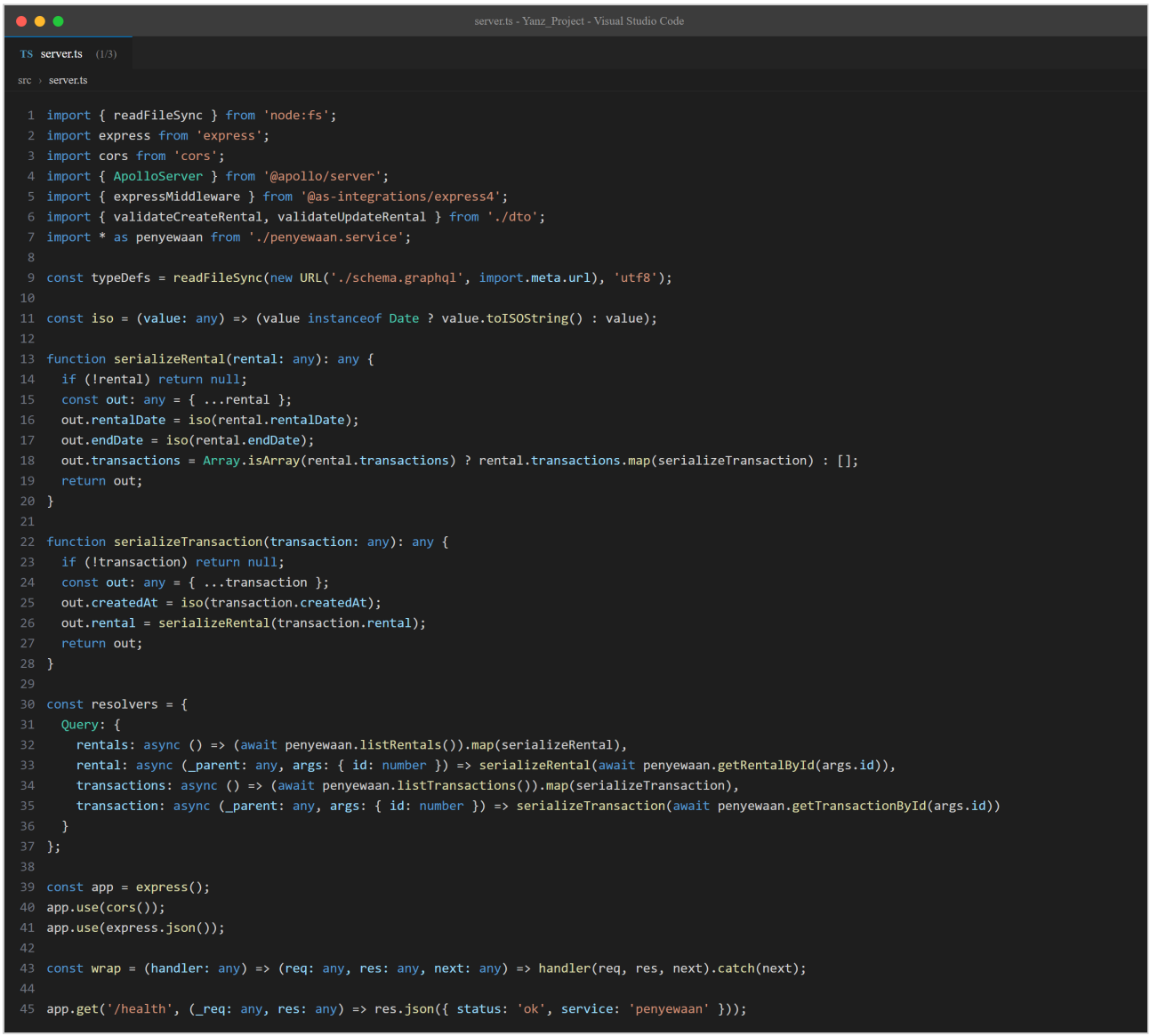
Struktur direktori proyek dibagi menjadi tiga bagian utama. Direktori prisma memuat skema basis data dan berkas seed. Direktori src memuat berkas validasi DTO, berkas layanan bisnis, definisi skema GraphQL, berkas server utama, halaman Apollo Sandbox dan playground, serta berkas pengujian asap (smoke test). Di tingkat akar terdapat berkas package.json yang mendaftarkan seluruh perintah operasional proyek serta berkas tsconfig.json yang mengatur konfigurasi kompilator.

3. Desain Endpoint REST dan Validasi DTO

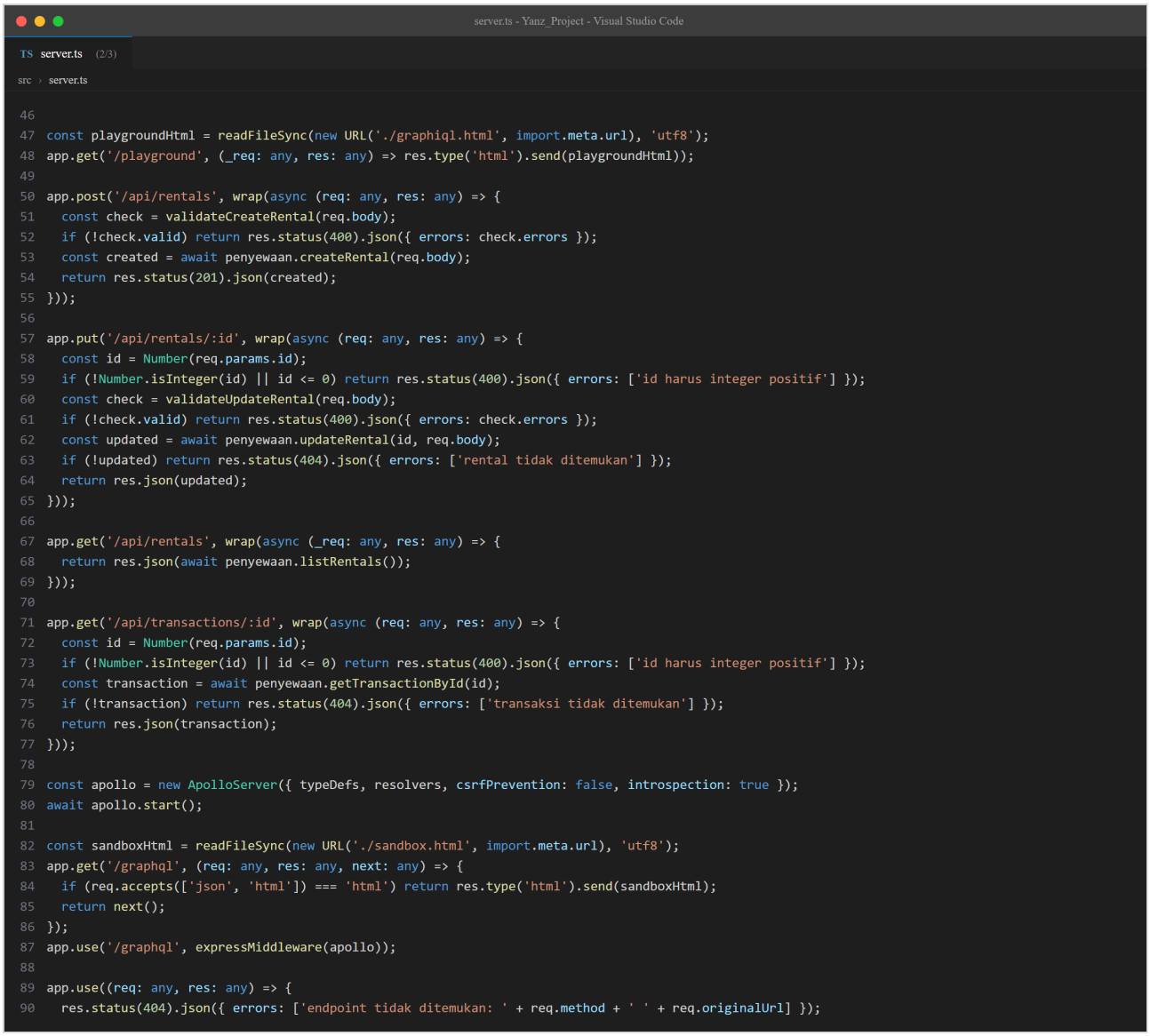
REST API dibangun dengan pendekatan resource-oriented. Setiap sumber daya memiliki alamat yang jelas dan menggunakan metode HTTP yang sesuai dengan semantik operasinya. Endpoint untuk membuat data menggunakan metode POST, endpoint untuk memperbarui data menggunakan metode PUT, sedangkan endpoint untuk membaca data menggunakan metode GET. Ringkasan endpoint disajikan pada tabel berikut.

Metode	URI	Fungsi
POST	/api/rentals	Membuat data penyewaan baru sekaligus mencatat transaksi pembayaran terkait
PUT	/api/rentals/:id	Memperbarui data penyewaan, misalnya kendaraan, tanggal, atau total biaya
GET	/api/rentals	Menampilkan seluruh data penyewaan beserta relasi pelanggan dan kendaraan
GET	/api/transactions/:id	Menampilkan detail transaksi beserta pelanggan dan penyewaan terkait

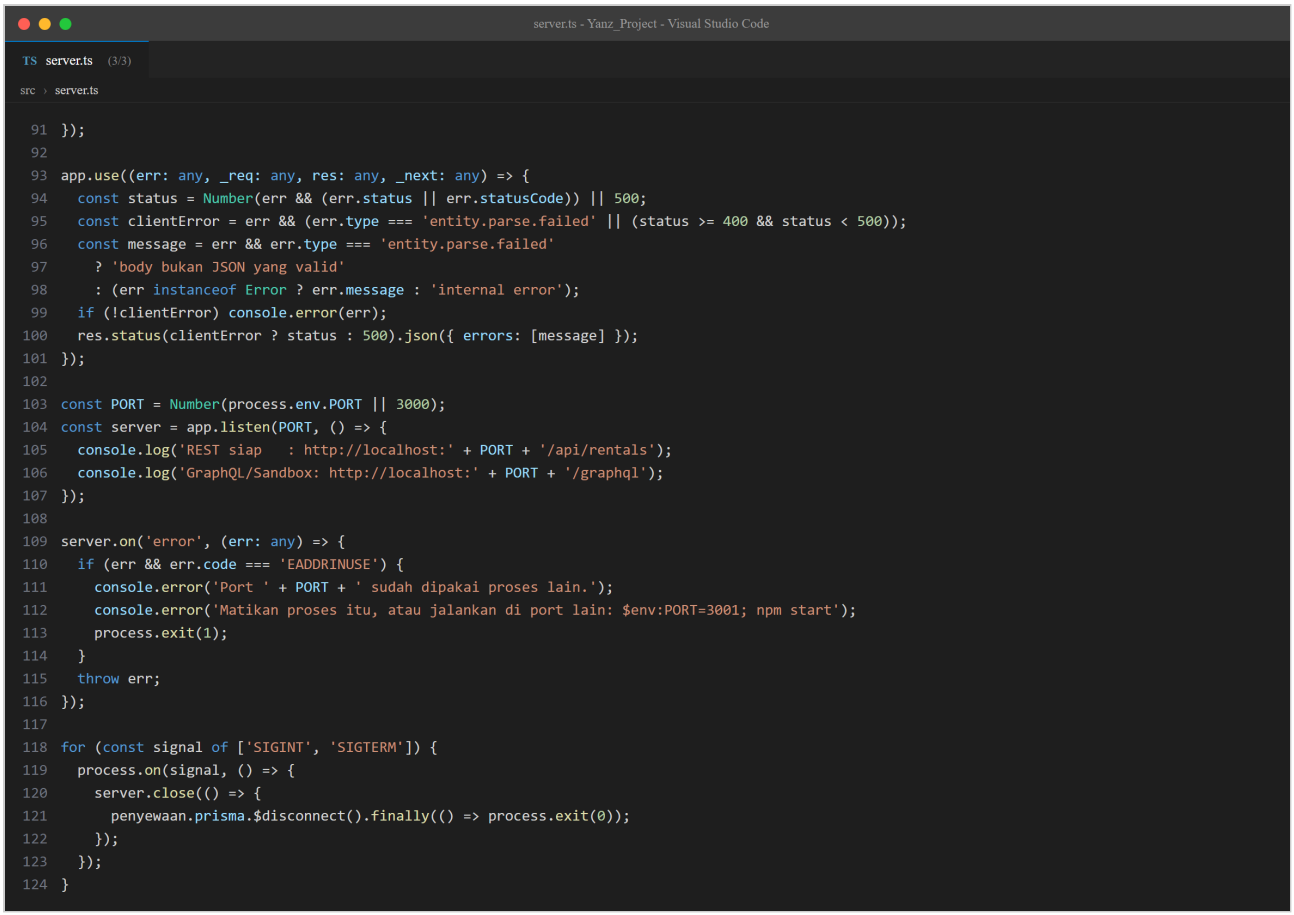
Implementasi keempat endpoint tersebut, penanganan kesalahan, pengaturan CORS, dan penyajian Apollo Sandbox terdapat pada berkas server.ts berikut.



Gambar 1. Berkas src/server.ts bagian 1 (baris 1-45): import, pembacaan skema GraphQL, serialisasi data, dan resolver.



Gambar 2. Berkas src/server.ts bagian 2 (baris 46-90): konfigurasi Express, CORS, endpoint REST, dan validasi masukan.

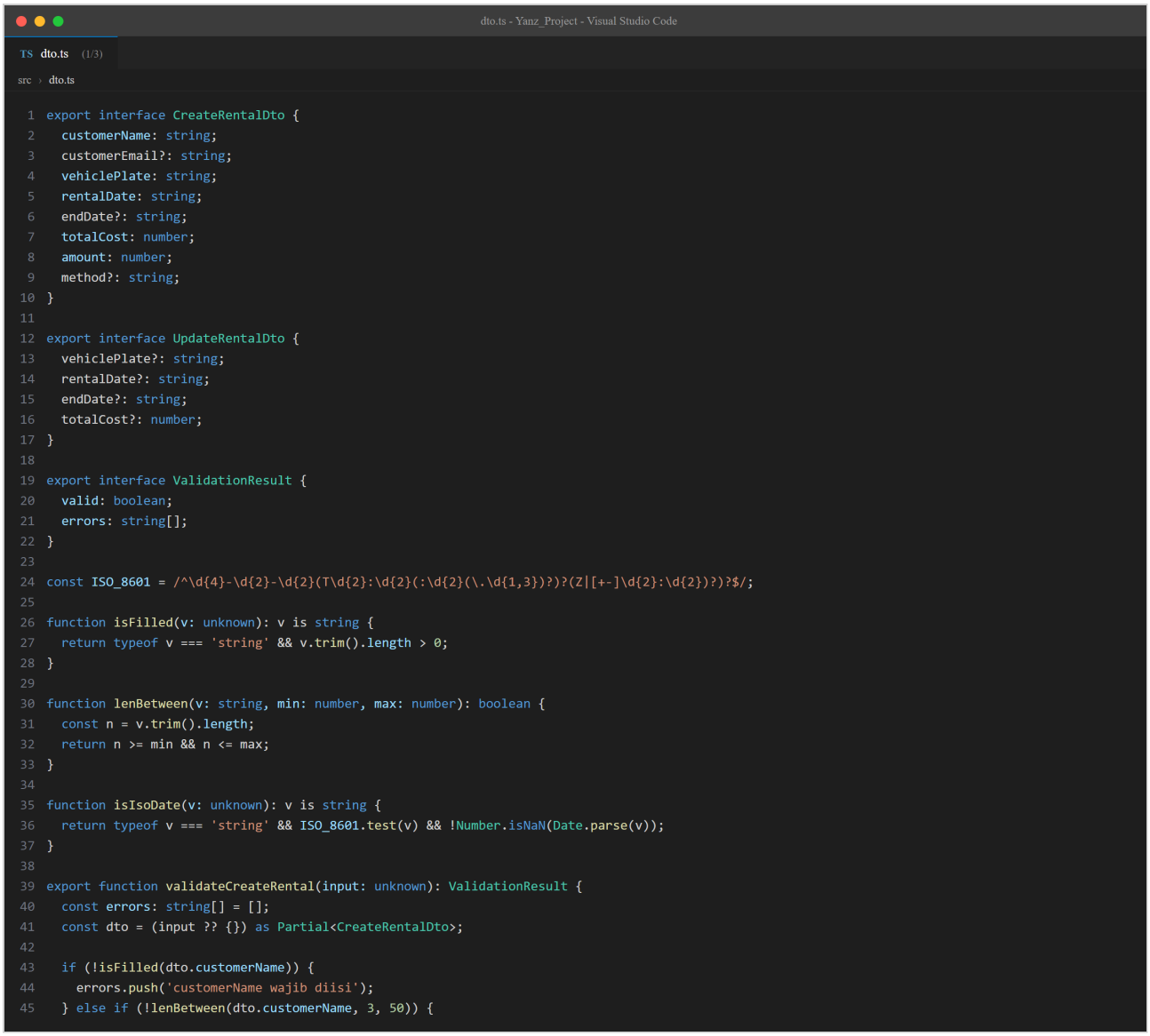


Gambar 3. Berkas src/server.ts bagian 3 (baris 91-124): Apollo Server, penyajian Sandbox, penanganan 404, dan penanganan error.

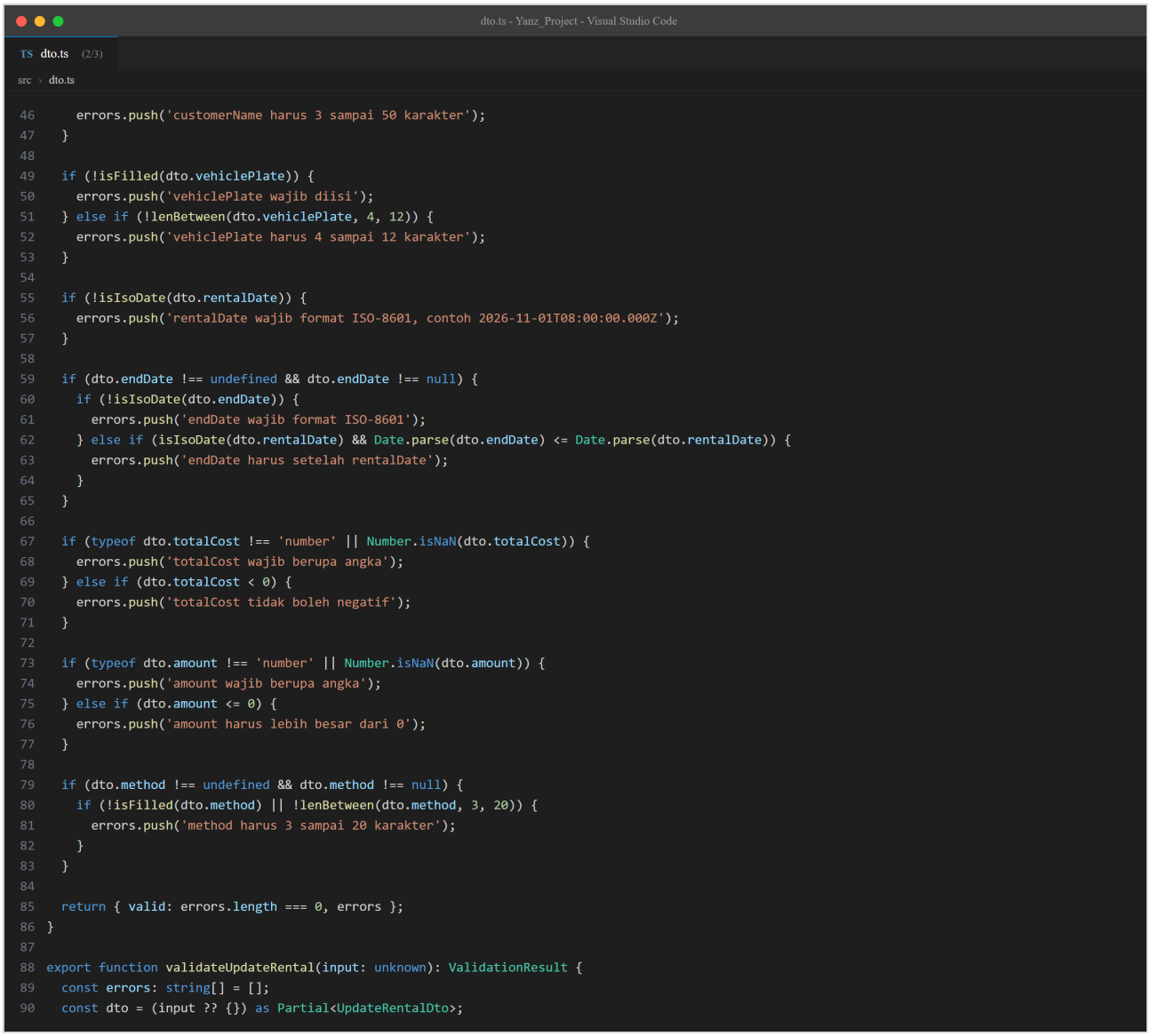
Sebelum data diproses oleh lapisan layanan, seluruh masukan terlebih dahulu divalidasi pada lapisan DTO (Data Transfer Object). Pendekatan ini memastikan bahwa data yang tidak valid tidak pernah mencapai basis data, sehingga integritas data tetap terjaga meskipun permintaan berasal dari klien yang tidak dapat dipercaya. Seluruh kesalahan validasi dikumpulkan terlebih dahulu, kemudian dikembalikan dalam satu respons sekaligus agar pengguna dapat memperbaiki semuanya dalam satu kali percobaan. Aturan validasi yang diterapkan adalah sebagai berikut.

Field	Aturan Validasi	Pesan Kesalahan
customerName	Wajib diisi, panjang 3 sampai 50 karakter	customerName wajib diisi / harus 3 sampai 50 karakter
vehiclePlate	Wajib diisi, panjang 4 sampai 12 karakter	vehiclePlate wajib diisi / harus 4 sampai 12 karakter
rentalDate	Wajib berformat ISO-8601	rentalDate wajib format ISO-8601
endDate	Opsional, ISO-8601, harus setelah rentalDate	endDate wajib format ISO-8601 / harus setelah rentalDate
totalCost	Wajib berupa angka dan tidak boleh negatif	totalCost wajib berupa angka / tidak boleh negatif
amount	Wajib berupa angka dan lebih besar dari nol	amount wajib berupa angka / harus lebih besar dari 0
method	Opsional, panjang 3 sampai 20 karakter	method harus 3 sampai 20 karakter

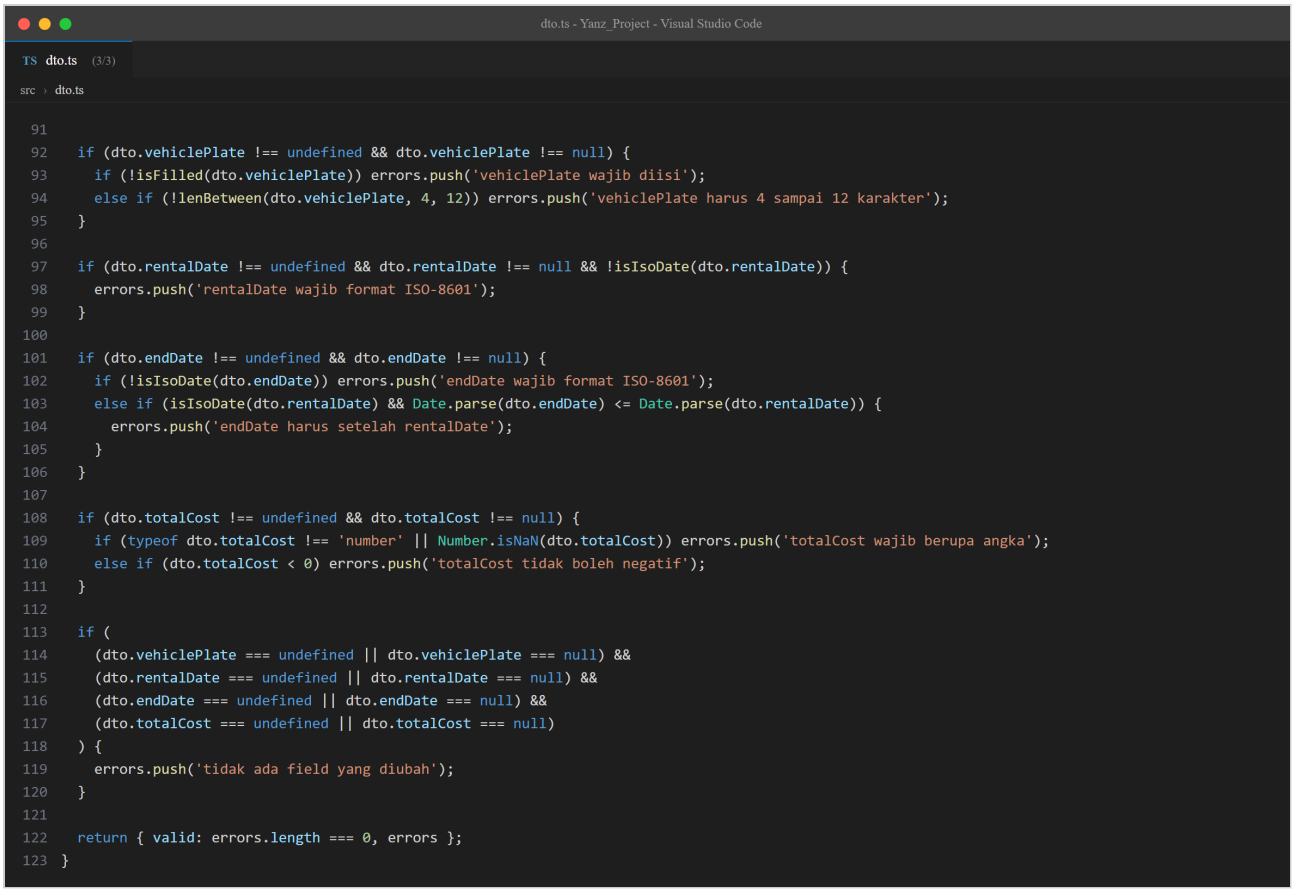
Apabila seluruh masukan valid, server membalas dengan kode 201 Created dan menyertakan data penyewaan beserta transaksi yang terbentuk. Sebaliknya, apabila terdapat masukan yang tidak valid, server membalas dengan kode 400 Bad Request beserta daftar kesalahan yang terperinci. Kode lengkap lapisan validasi ditunjukkan pada gambar berikut.



Gambar 4. Berkas src/dto.ts bagian 1 (baris 1-45): definisi tipe DTO dan fungsi bantu validasi.



Gambar 5. Berkas src/dto.ts bagian 2 (baris 46-90): validasi pembuatan penyewaan.



Gambar 6. Berkas src/dto.ts bagian 3 (baris 91-123): validasi pembaruan penyewaan.

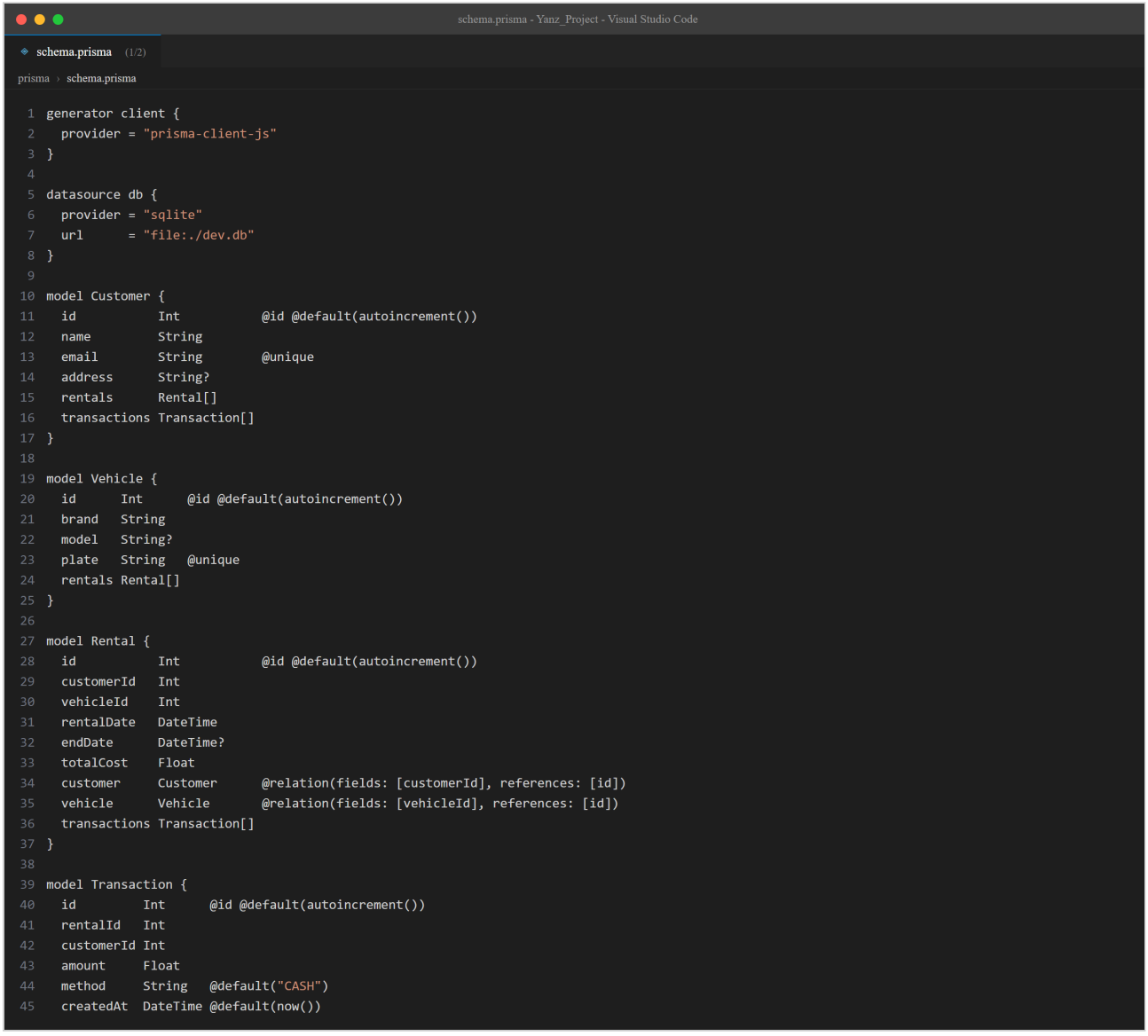
4. Model Basis Data dan Relasi

Basis data dirancang menggunakan empat model utama. Model Customer menyimpan identitas pelanggan, model Vehicle menyimpan data kendaraan, model Rental menyimpan transaksi penyewaan, dan model Transaction menyimpan catatan pembayaran. Pemilihan nama model mengikuti kaidah penamaan tunggal (singular) yang merupakan konvensi umum pada Prisma.

Relasi antar model dibangun dengan memperhatikan kaidah normalisasi. Tabel Rental menyimpan kunci asing customerId dan vehicleId, sehingga satu pelanggan dapat memiliki banyak penyewaan dan satu kendaraan dapat disewa berkali-kali pada waktu yang berbeda. Tabel Transaction menyimpan kunci asing rentalId dan customerId, sehingga satu penyewaan dapat memiliki beberapa transaksi, misalnya pembayaran uang muka dan pelunasan.

Model	Field Utama	Relasi
Customer	id, name, email (unik), address	1 - N ke Rental, 1 - N ke Transaction
Vehicle	id, brand, model, plate (unik)	1 - N ke Rental
Rental	id, customerId, vehicleId, rentalDate, endDate, totalCost	N - 1 ke Customer, N - 1 ke Vehicle, 1 - N ke Transaction
Transaction	id, rentalId, customerId, amount, method, createdAt	N - 1 ke Rental, N - 1 ke Customer

Tipe data yang digunakan juga disesuaikan dengan karakteristik masing-masing field. Tanggal menggunakan tipe DateTime, nominal biaya menggunakan tipe Float, dan kolom email serta plat nomor diberi atribut unik untuk mencegah duplikasi data. Skema lengkap yang digunakan adalah sebagai berikut.



Gambar 7. Berkas prisma/schema.prisma: definisi model Customer, Vehicle, Rental, dan Transaction beserta relasinya.

5. Lapisan Layanan (Service Layer)

Seluruh logika bisnis ditempatkan pada berkas penyewaan.service.ts dan tidak dicampur dengan kode penanganan HTTP. Pemisahan ini memberikan beberapa keuntungan sekaligus. Pertama, kode menjadi lebih mudah diuji karena fungsi layanan dapat dipanggil secara langsung tanpa memerlukan server HTTP. Kedua, kode menjadi lebih mudah digunakan ulang, karena endpoint REST dan resolver GraphQL dapat memanggil fungsi layanan yang sama sehingga perilaku keduanya konsisten.

Fungsi utama pada lapisan ini adalah getTransactionById yang mengambil satu transaksi beserta data pelanggan dan data penyewaan terkait, termasuk kendaraan yang disewa. Selain itu tersedia fungsi getRentalById, listRentals, listTransactions, createRental, dan updateRental. Fungsi createRental melakukan beberapa langkah sekaligus, yaitu mencari atau membuat data pelanggan berdasarkan email, mencari atau membuat data kendaraan berdasarkan plat nomor, membuat data penyewaan, lalu membuat data transaksi pembayaran.

Pendekatan upsert pada proses pembuatan data pelanggan dan kendaraan bertujuan agar sistem tidak membuat data ganda ketika pelanggan yang sama melakukan penyewaan lebih dari satu kali. Dengan demikian, data pelanggan dan kendaraan tetap konsisten sepanjang siklus hidup aplikasi.

penyewaan.service.ts - Yanz_Project - Visual Studio Code

TS penyewaan.service.ts (1/3)

src › penyewaan.service.ts

```ts
import { PrismaClient } from '@prisma/client';
import type { CreateRentalDto, UpdateRentalDto } from './dto';

export const prisma = new PrismaClient();

const DEFAULT_BRAND = 'Honda';
const DEFAULT_MODEL = 'Vario 160';

function normalizePlate(plate: string): string {
  return plate.trim().toUpperCase().split(' ').join(' ');
}

function slugEmail(name: string): string {
  return name.trim().toLowerCase().split(' ').join('.') + '@example.com';
}

async function resolveVehicle(plate: string) {
  const clean = normalizePlate(plate);
  const found = await prisma.vehicle.findUnique({ where: { plate: clean } });
  if (found) return found;
  return prisma.vehicle.create({ data: { plate: clean, brand: DEFAULT_BRAND, model: DEFAULT_MODEL } });
}

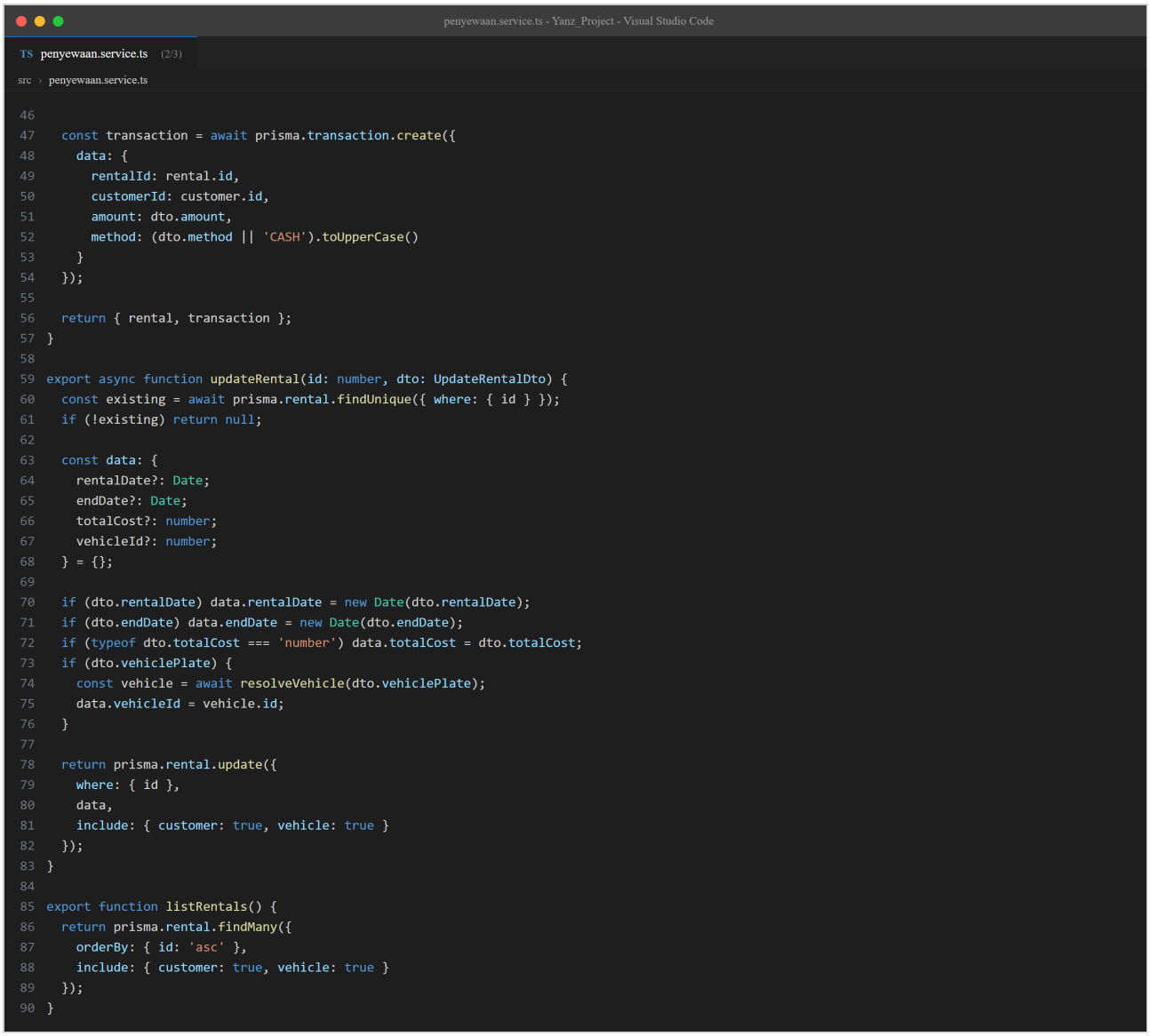
export async function createRental(dto: CreateRentalDto) {
  const customerName = dto.customerName.trim();
  const email = (dto.customerEmail || '').trim() || slugEmail(customerName);

  const customer = await prisma.customer.upsert({
    where: { email },
    update: { name: customerName },
    create: { name: customerName, email }
  });

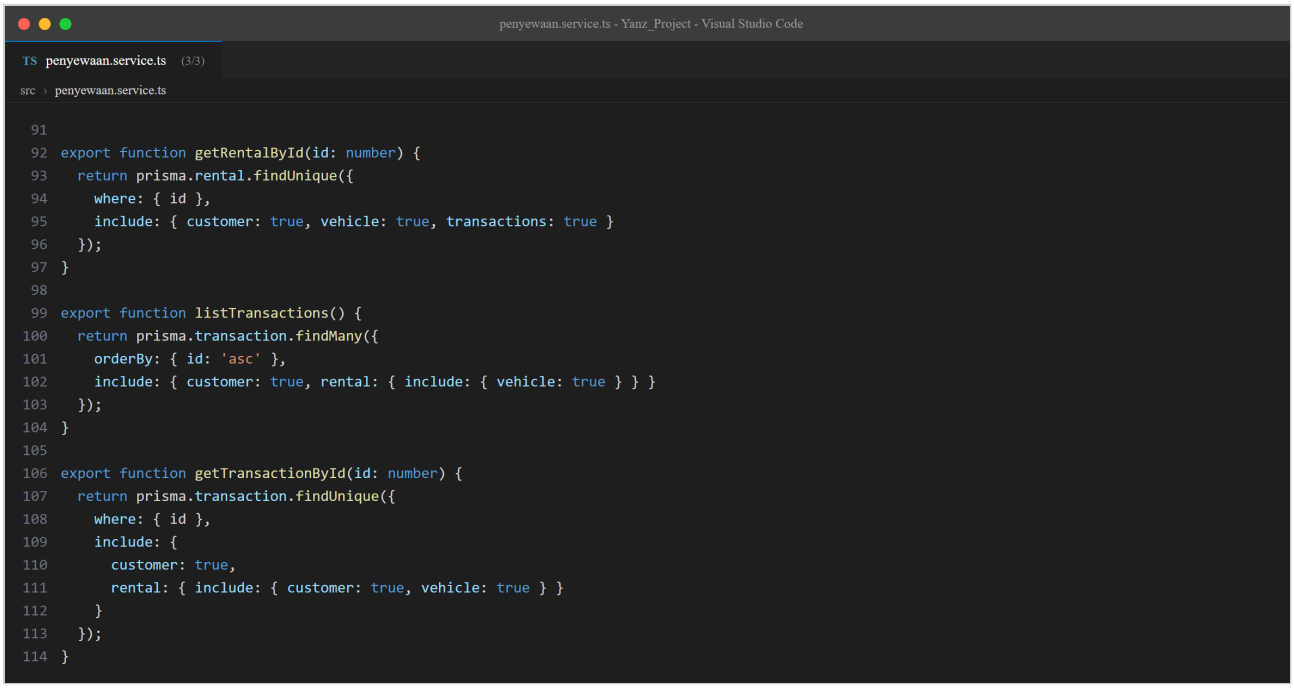
  const vehicle = await resolveVehicle(dto.vehiclePlate);

  const rental = await prisma.rental.create({
    data: {
      customerId: customer.id,
      vehicleId: vehicle.id,
      rentalDate: new Date(dto.rentalDate),
      endDate: dto.endDate ? new Date(dto.endDate) : null,
      totalCost: dto.totalCost
    },
    include: { customer: true, vehicle: true }
  });
```

Gambar 8. Berkas src/penyewaan.service.ts bagian 1 (baris 1-45): inisialisasi Prisma, fungsi bantu, dan pembuatan penyewaan.



Gambar 9. Berkas src/penyewaan.service.ts bagian 2 (baris 46-90): pembaruan penyewaan dan fungsi pembacaan data.

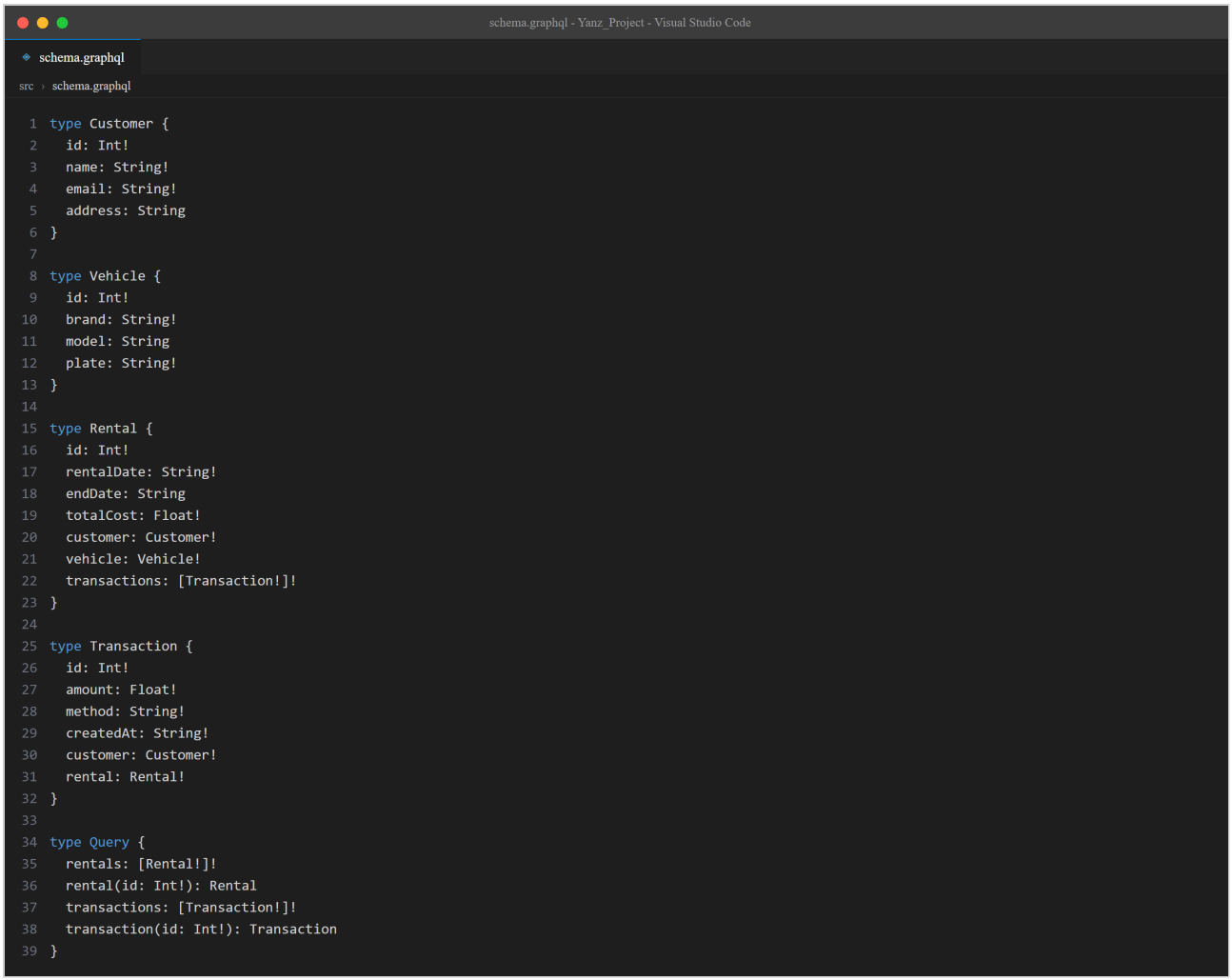


Gambar 10. Berkas src/penyewaan.service.ts bagian 3 (baris 91-114): pengambilan transaksi beserta relasi pelanggan dan penyewaan.

6. GraphQL API dan Apollo Sandbox

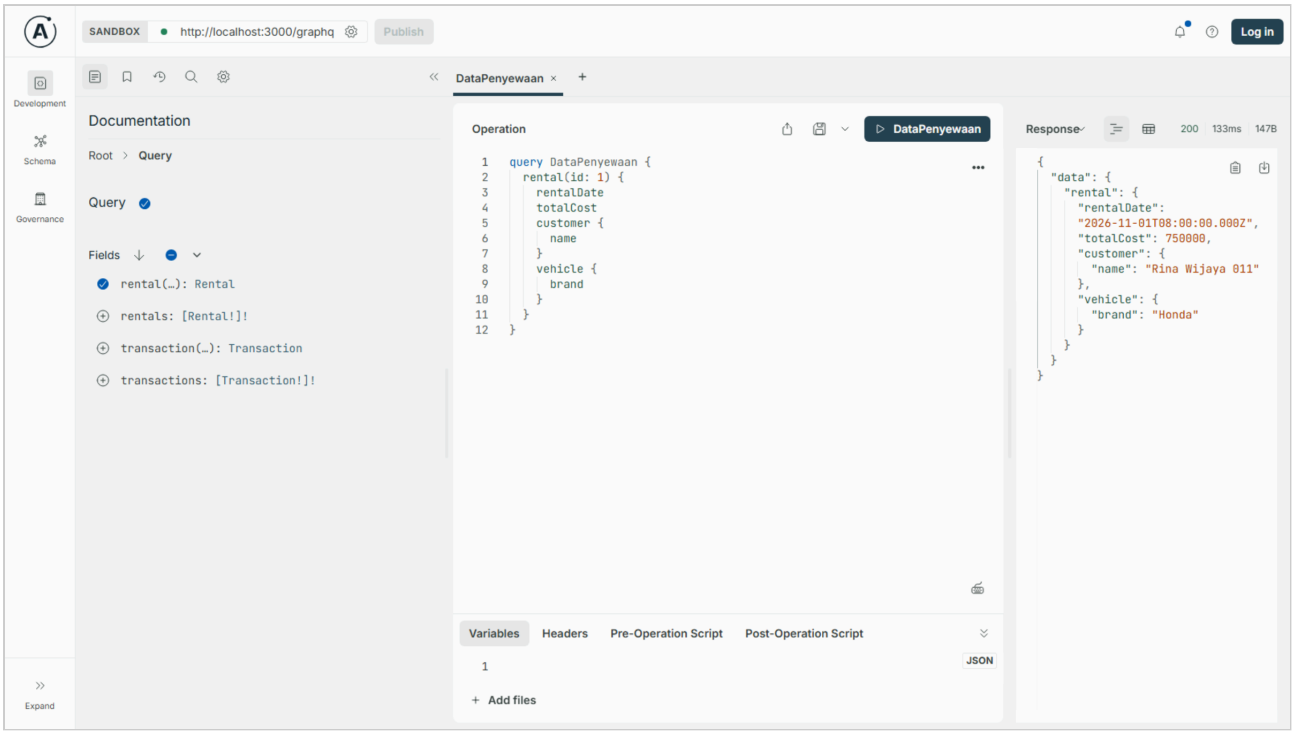
GraphQL disediakan sebagai alternatif pengambilan data. Berbeda dengan REST yang mengembalikan struktur respons yang telah ditentukan oleh server, GraphQL memungkinkan klien menentukan sendiri field yang dibutuhkan. Hal ini mengurangi jumlah data yang dikirim melalui jaringan serta mengurangi kebutuhan untuk melakukan beberapa permintaan hanya demi melengkapi satu tampilan.

Skema GraphQL yang dibangun memetakan setiap model Prisma menjadi sebuah tipe objek. Tipe Rental memiliki field customer dan vehicle yang bertipe objek, sehingga relasi antar tabel dapat ditelusuri dalam satu permintaan. Tipe Query menyediakan empat operasi pembacaan, yaitu rentals, rental, transactions, dan transaction.



Gambar 11. Berkas src/schema.graphql: definisi tipe Customer, Vehicle, Rental, Transaction, dan Query.

Agar pengujian manual lebih mudah, alamat /graphql menyajikan Apollo Sandbox yang langsung terhubung ke endpoint yang sama dan sudah terisi query contoh yang valid, sehingga penguji cukup menekan tombol Run. Server juga mengirimkan header CORS dan menjawab preflight OPTIONS, sehingga Sandbox dapat mengirim permintaan ke endpoint meskipun berada pada origin yang berbeda. Query yang diuji beserta hasilnya ditunjukkan pada gambar berikut.



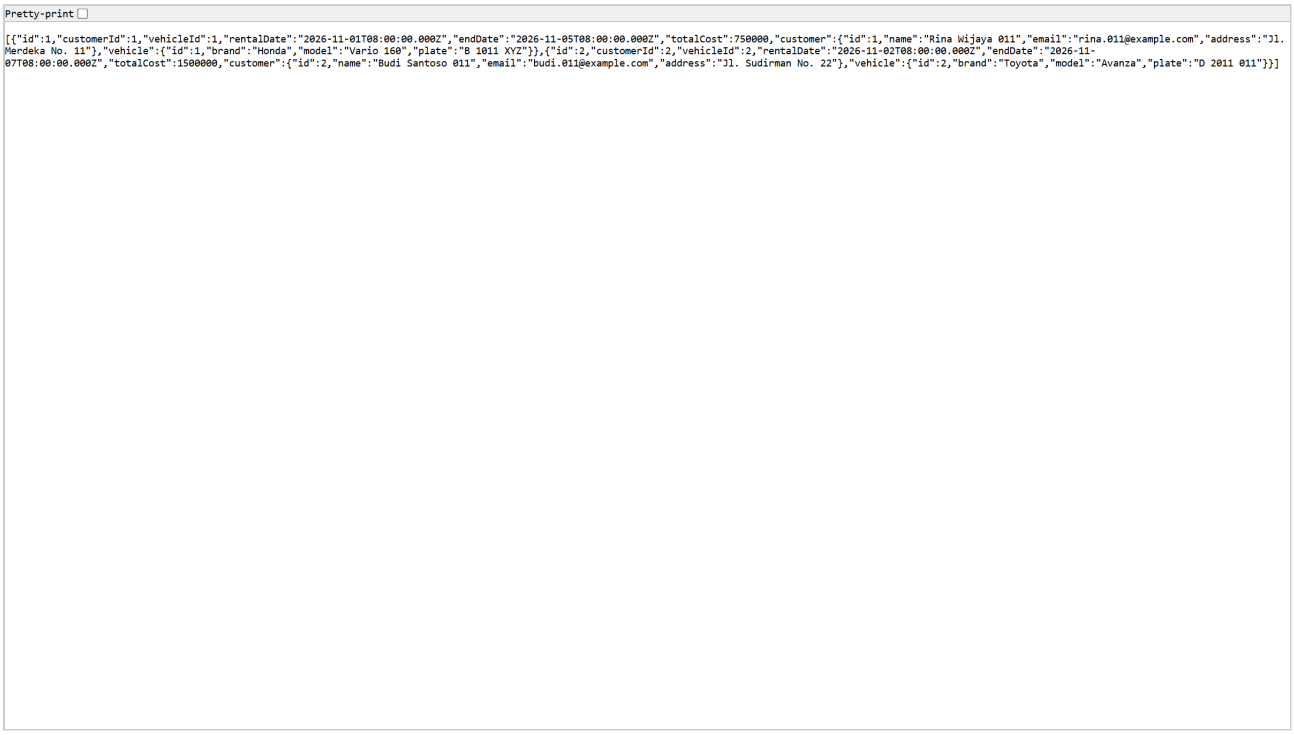
Gambar 12. Apollo Sandbox menjalankan query DataPenyewaan dan menerima respons 200 OK berisi data penyewaan, pelanggan, dan kendaraan.

7. Pengujian dan Hasil

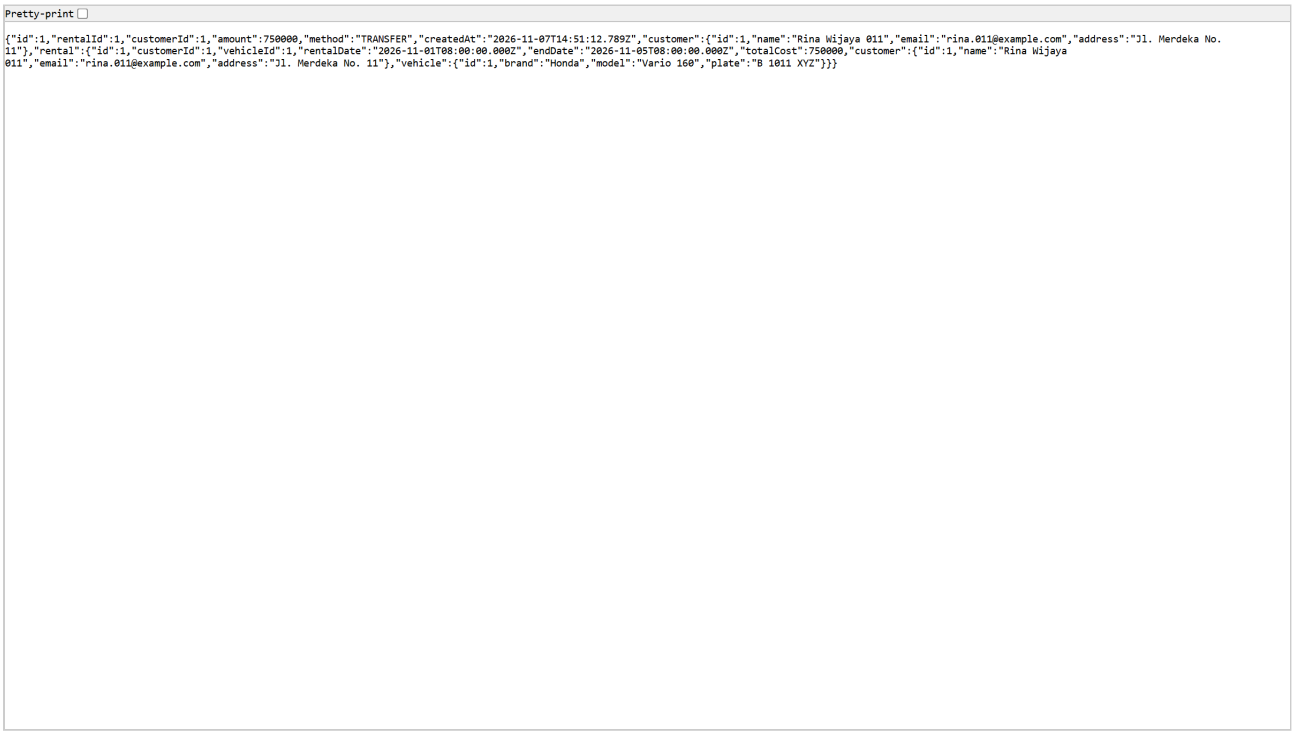
Pengujian dilakukan dengan dua pendekatan. Pertama, pengujian manual dengan mengirim permintaan HTTP langsung ke setiap endpoint untuk memastikan respons sesuai harapan. Kedua, pengujian otomatis melalui skrip smoke test yang dijalankan dengan perintah npm run smoke. Skrip ini memeriksa status kode, struktur respons, keterkaitan relasi, serta memastikan bahwa masukan tidak valid benar-benar ditolak oleh lapisan validasi.

Skenario Pengujian	Hasil yang Diharapkan	Status
Membuat penyewaan dengan data valid	201 Created, data tersimpan	Lulus
Membuat penyewaan dengan data tidak valid	400 Bad Request, daftar kesalahan lengkap	Lulus
Memperbarui penyewaan yang ada	200 OK, data berubah	Lulus
Memperbarui penyewaan tanpa field	400 Bad Request	Lulus
Memperbarui penyewaan yang tidak ada	404 Not Found	Lulus
Mengirim body JSON yang rusak	400 Bad Request	Lulus
Mengakses endpoint yang tidak terdaftar	404 Not Found berformat JSON	Lulus
Query GraphQL rental dengan relasi	200 OK, seluruh field terisi	Lulus
Query GraphQL tidak valid	400 Bad Request, pesan kesalahan	Lulus

Hasil pengujian menunjukkan bahwa seluruh skenario berjalan sesuai harapan. Berkas pengujian otomatis diakhiri dengan keluaran SMOKE OK dan kode keluar nol, yang menandakan tidak ada kegagalan. Pemeriksaan tipe dengan kompilator TypeScript juga selesai tanpa kesalahan, dan pemeriksaan dependensi menunjukkan tidak ada kerentanan yang dilaporkan. Bukti respons endpoint ditunjukkan pada gambar berikut.



Gambar 13. Respons endpoint GET /api/rentals yang menampilkan data pelanggan dan kendaraan.



Gambar 14. Respons endpoint GET /api/transactions/1 yang menampilkan relasi transaksi, pelanggan, penyewaan, dan kendaraan.

8. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan pada Ujian Tengah Semester ini, sistem penyewaan kendaraan berhasil dibangun sesuai dengan kebutuhan yang ditetapkan. Lapisan REST API menyediakan operasi pembacaan dan penulisan data dengan validasi masukan yang ketat, lapisan basis data menggunakan skema relasional yang ternormalisasi, dan lapisan layanan memisahkan logika bisnis dari kode penanganan HTTP sehingga struktur program menjadi lebih rapi dan mudah dikembangkan.

Penambahan GraphQL API memberikan fleksibilitas lebih bagi klien dalam menentukan data yang dibutuhkan, terutama ketika data tersebut tersebar pada beberapa tabel yang saling berelasi. Seluruh pengujian fungsional berhasil dilalui tanpa kesalahan, sehingga sistem dinyatakan siap digunakan dan siap didemonstrasikan sebagai hasil pekerjaan UTS.

Lampiran - Cara Menjalankan Program

- npm install - memasang seluruh dependensi proyek.
- npm run db:reset - membuat basis data SQLite dan mengisi data contoh.
- npm start - menjalankan server pada http://localhost:3000.
- npm run start:dev - menjalankan server dalam mode watch.
- npm run smoke - menjalankan pengujian otomatis seluruh endpoint.

Alamat penting: REST API http://localhost:3000/api/rentals, Apollo Sandbox http://localhost:3000/graphql, dan playground lokal http://localhost:3000/playground.